Lesson Breakdown:

1. Introduction to Linked Lists (10-15 minutes)

1. **The Concept**:
    - What is a Linked List?
        - A dynamic data structure consisting of nodes.
        - Each node contains data and a pointer to the next node.
    - Types of Linked Lists:
        - Singly Linked List
        - Doubly Linked List
        - Circular Linked List
    - Advantages over arrays:
        - Dynamic size
        - Efficient insertion and deletion
2. **Visualization**:
    - Use diagrams to show how nodes are linked in memory:

```
[Data|Next] -> [Data|Next] -> [Data|Next] -> nullptr
```

2. Create a Node Class (20 minutes)

- **Step 1: Define a `Node` Class**:
    - Explain how to represent a node in C++.

```
class Node {
public:
    int data;       // To store data
    Node* next;     // Pointer to the next node

    Node(int data) {
        this->data = data;
        this->next = nullptr;
    }
};
```

- **Hands-on Activity**:
    - Students need to write and compile this class in their IDE.
    - Verify that the student understands the constructor.

3. Building the LinkedList Class (30 minutes)

- **Step 2: Define a `LinkedList` Class**:
    - Introduce the `LinkedList` class with a `head` pointer.

```
class LinkedList {
private:
    Node* head;

public:
    LinkedList() {
        head = nullptr;
    }
```

```
};
```

- **Step 3: Adding Nodes to the List**:
    - How to append nodes to the linked list.

```
void addNode(int data) {
    Node* newNode = new Node(data);
    if (head == nullptr) {
        head = newNode;  // First node
    } else {
        Node* current = head;
        while (current->next != nullptr) {
            current = current->next;  // Traverse to the last node
        }
        current->next = newNode;  // Link the new node
    }
}
```

- **Step 4: Traversing the List**:
    - How to iterate through the linked list to print data.

```
void printList() {
    Node* current = head;
    while (current != nullptr) {
        std::cout << current->data << " -> ";
        current = current->next;
    }
    std::cout << "nullptr" << std::endl;
}
```

- **Hands-on Activity**:
    - Students need to implement these methods in the `LinkedList` class.
    - Guide students through compiling and testing.

4. Demonstrating with `main()` (20 minutes)

- **Step 5: Writing the `main()` Function**:

```
int main() {
    LinkedList list;

    list.addNode(10);
    list.addNode(20);
    list.addNode(30);

    list.printList();  // Expected Output: 10 -> 20 -> 30 -> nullptr

    return 0;
}
```

- **Hands-on Activity**:
    - Students should implement the `main()` function.
    - Test the linked list by adding and printing nodes.

5. Advanced Operations (Optional, 15 minutes)

- **Step 6: Deleting a Node**:
    - Introduce deletion (e.g., delete from the front, middle, or end).

```
void deleteNode(int key) {
    Node* current = head;
    Node* prev = nullptr;

    if (current != nullptr && current->data == key) {
        head = current->next;  // Key found at head
        delete current;
        return;
    }

    while (current != nullptr && current->data != key) {
        prev = current;
        current = current->next;
    }

    if (current == nullptr) return;  // Key not found

    prev->next = current->next;  // Unlink the node
    delete current;
}
```

- **Step 7: Destructor to Free Memory**:

```
~LinkedList() {
    Node* current = head;
    while (current != nullptr) {
        Node* next = current->next;
        delete current;
        current = next;
    }
}
```

6. Recap and Q&A (10 minutes)

- Summarize the key points:
    - Node creation
    - Adding and traversing nodes
    - Advanced operations (deletion, destructor)
- Students can ask questions.